

基于变异和约束求解的程序缺陷自动修复方法

董 兰, 洪 玫⁺, 伍 佳

(四川大学 计算机学院 (软件学院), 四川 成都 610065)

摘 要: 为能正确高效地生成修复补丁, 针对 Java 程序中出现频率较高的条件语句相关缺陷修复问题, 将启发式搜索方法与语义约束求解方法相结合, 提出一个有针对性、更高效的解决方案。针对条件语句缺失错误, 采用基于组件的程序合成技术, 合成满足约束的候选条件语句; 针对条件语句逻辑表达式错误, 采用变异技术, 生成候选逻辑表达式; 针对条件语句逻辑表达式错误中, 不能用变异技术修复的缺陷, 使用基于组件的约束求解方法生成候选补丁。实验结果表明, 所提方法有更高的补丁召回率和准确率。

关键词: 程序自动修复; 变异分析; 约束求解; 程序合成; 条件语句缺陷; 补丁生成; 软件调试

中图法分类号: TP311 **文献标识号:** A **文章编号:** 1000-7024 (2024) 01-0088-07

doi: 10.16208/j.issn1000-7024.2024.01.012

Automatic repair method of program defects based on variation and constraint solving

DONG Lan, HONG Mei⁺, WU Jia

(College of Computer Science (College of Software), Sichuan University, Chengdu 610065, China)

Abstract: To generate repair patches correctly and efficiently, a targeted and more efficient method was proposed by combining the heuristic search method with the semantic constraint solving method for automatic repair of buggy conditional statements with high frequency in Java programs. Aiming at the missing precondition bugs of conditional statements, the component-based program synthesis technology was used to generate candidate conditional statements that satisfied the constraints. At the same time, for the buggy conditional statements, mutation technology was used to generate candidate logical expressions. For the buggy conditional statements that could not be repaired using mutation techniques, component-based constraint solving method was used to generate candidate patches. Experimental results demonstrate that the proposed method has higher patch recall and accuracy.

Key words: automatic program repair; mutation analysis; constraint solving; program synthesis; conditional statement bugs; patch generation; software debugging

0 引 言

目前软件缺陷自动修复技术^[1]主要有基于启发式搜索、语义约束、人工修复模板、统计分析等方法, 其中大部分方法都通用的, 缺陷修复的召回率和准确率不高^[2]。对于基于人工修复模板的方法具有针对性, 只能针对常见、特征明确的缺陷类型。而基于测试的缺陷修复方法则依赖于通过的测试数据, 而在 Defects4J 库中有 95% 以上的程序

缺陷在被修复之前没有相应的未通过测试, 而测试覆盖率与补丁质量成正相关, 挖掘到的程序规约信息有限, 约束求解效果不好^[2]。

本文针对 Java 程序中出现频率较高的条件语句的相关缺陷修复问题, 将启发式搜索方法与语义约束求解方法相结合, 提出一个更高效的解决方案。针对条件语句缺失错误, 基于约束求解, 通过收集程序语义信息, 采用基于组件的程序合成技术, 生成候选条件表达式。同时, 针对条

收稿日期: 2022-04-18; 修订日期: 2023-12-27

基金项目: 国家重点研发计划基金项目 (2020YFB1711801)

作者简介: 董兰 (1997-), 女, 四川阿坝州人, 硕士研究生, 研究方向为软件质量保证与测试; ⁺通讯作者: 洪玫 (1963-), 女, 浙江舟山人, 硕士, 教授, 研究方向为软件分析与测试; 伍佳 (1996-), 女, 四川成都人, 硕士研究生, 研究方向为软件质量保证与测试。

E-mail: 2388027092@qq.com

件语句逻辑表达式错误, 首先采用变异技术, 生成候选补丁; 其次, 对于条件语句逻辑表达式错误中不能利用变异技术修复的缺陷, 再使用约束求解方法生成补丁。最后验证补丁的有效性。该方案解决了 Java 程序自动修复中的重要问题, 并在修复效率上有所提高, 值得借鉴和参考。

1 相关工作

在自动软件缺陷修复技术中, 常用的修复方法是基于启发式搜索的技术, 该方法通过改变原始缺陷程序生成新程序作为候选补丁。代表性研究有 GenProg、RSRepair、Kail 等方法。GenProg^[3]采用抽象语法树、遗传算法, 运用交叉、变异算子等生成程序候选补丁。RSRepair^[4]选择随机搜索来指导生成-验证过程, 而 Kail^[5]是一种试图通过删除潜在的不必要但有害的功能来修复程序的技术。

基于语义约束的缺陷修复方法, 其综合应用缺陷定位、天使调试、约束求解、程序合成等多种技术^[6]来修复补丁, 代表性研究有 SemFix、Angelix 和 Nopol 等方法。SemFix^[7-9]综合使用天使调试和基于组件的程序合成算法, 约束求解器求解获得程序补丁。Angelix^[10]旨在保持可伸缩性的同时综合多行修复。而 Nopol^[11,12]合成了在条件或循环语句中发生的一条更改条件组成的修复, 利用天使修复定位获得天使值, 通过合成条件表达式修复缺陷。

针对 Java 程序缺陷的修复方法, 除了 Nopol 方法之外, 还有针对修复 Java 语言的内存溢出、资源泄露、空指针引用、内存溢出等缺陷的 FootPatch 方法^[13], 以及高精度的条件语句综合技 ACS^[14], 针对空指针、数据越界以及类型强转缺陷的 Genesis 自动修复技术^[15]。

由于单一的自动缺陷修复方法不能保证任何类型的缺陷都可以成功修复^[16], 因此, 针对特定缺陷类型, 考虑综合的自动化修复方法就有一定的研究价值。本文针对 Java 程序中条件语句逻辑错误和条件语句缺失错误两类缺陷, 借鉴启发式搜索中的变异方法和语义驱动的约束求解、组件合成等方法, 并对这些方法进行综合运用, 以提高缺陷修复率和补丁精度。

2 问题描述和基本思路

研究者通过挖掘和分析 7 个大型开源 Java 项目的历史缺陷修复记录, 总结了 9 大类、27 个缺陷修复模式。其中, 最常见的是 IF 条件相关 (IF), 占 19.7%~33.9%^[17]。而 IF-CC 和 IF-APC 所对应的条件语句错误和条件语句缺失两类缺陷在实际缺陷中占据较大比例, 见表 1。

因此针对 Java 程序中的条件语句逻辑错误和条件语句缺失错误的常见缺陷, 探索缺陷自动修复方法。条件语句逻辑表达式错误是指条件表达式存在逻辑错误, 导致程序无法执行到期望的分支; 条件语句缺失错误是指在执行某些操作之前, 缺少前置条件的检查, 比如检测空指针。

表 1 Java 程序中的缺陷修复模式

分类	修复模式
If-related (IF)	添加前置条件 (IF-APC)
	添加带有跳转的前置条件 (IF-APCJ)
	添加后置条件 (IF-APTC)
	移除 IF 语句 (IF-RMV)
	添加 ELSE 分支 (IF-ABR)
	移除 ELSE 分支 (IF-RBR)
	修改 IF 条件表达式 (IF-CC)
Switch (SW)	增加/移除 Switch 分支 (SW-ARSB)
Loop (LP)	修改循环的前置条件 (LP-CC)
	修改用于改变循环变量值的表达式 (LP-CE)

针对上述两类条件语句相关的缺陷, 本文采用收集测试信息 (G1), 使用本文提出的方法生成满足的候选补丁 (G2), 最后通过测试套件验证候选补丁 (G3) 这 3 个阶段过程, 实现缺陷的自动修复。首先通过执行相关的测试用例, 收集程序执行时待修复程序位置处可访问的数据变量和对应的值, 形成待求解的修复方程式; 其次, 确定可修复程序位置的语句类型以采用不同的补丁生成方案进行缺陷修复。如果待修复位置是条件语句表达式缺陷, 先采用变异技术产生候选补丁, 如果没有相符合的补丁, 再使用基于约束求解方法, 合成条件表达式; 如果是条件语句缺失缺陷, 直接应用基于约束求解方法生成表达式, 并将生成的表达式作为前置条件添加在源程序中。最后, 运行测试用例, 再次进行补丁验证。整体方案流程如图 1 所示。

3 基于测试数据形成缺陷修复方程式

根据测试用例的执行, 收集与待修复缺陷相关的测试用例执行过程中的上下文信息。

3.1 条件表达式实际输入数据收集

在程序执行过程中, 收集执行到待修复条件语句 *exp-patch* 之前所有可访问的变量以及变量值, 包括基本类型数据和面对对象程序的特定数据, 主要包括局部变量、成员变量、方法参数以及它们对应的值。将收集到的数据称为输入数据, 用 $I_{l,m,n}$, 其中, l 表示待修复条件表达式 *exp-patch* 所在位置, n 和 m 分别表示第 n 个测试用例的第 m 次执行。除此之外, 还增加 3 个基准常数 $\{-1, 0, 1\}$ 来丰富 $I_{l,m,n}$, 其它的数据都可以基于这 3 个基准常数构建。假设 l 位置在可修复的程序位置范围内有 a 个基本变量和一组对象集合 O (共 O 个对象), 则 $I_{l,m,n}$ 包含的数据如下:

- a 个基本变量以及变量对应的变量值。如: $a \rightarrow 10$;
- w 个 *boolean* 值, 分别表示 O 个对象是否为空。如: $obj1 = null \rightarrow false$;

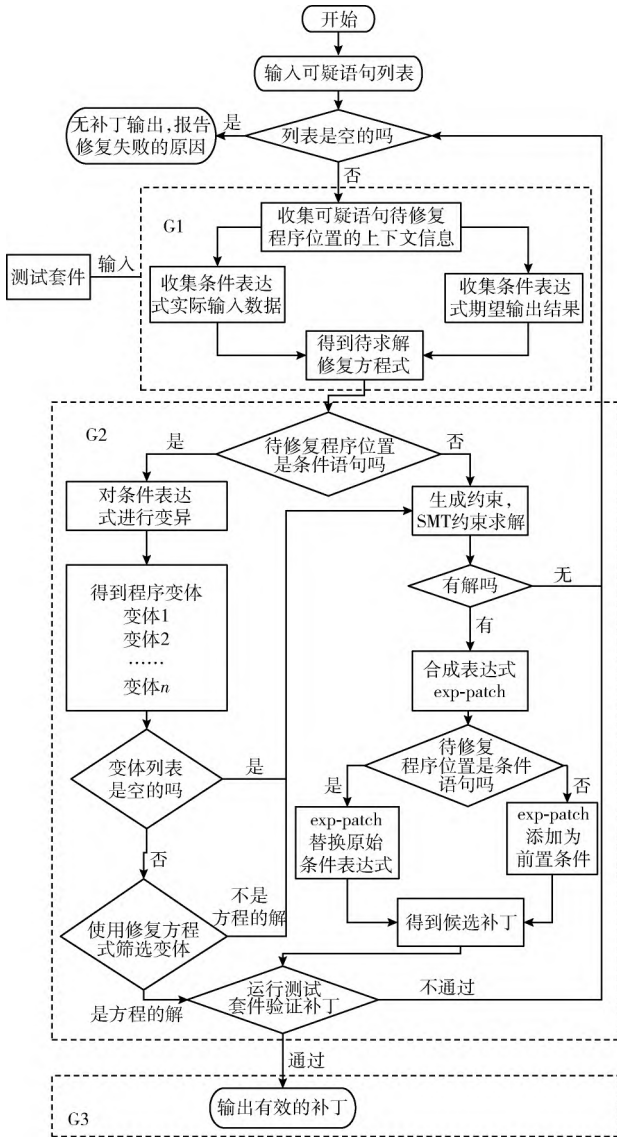


图 1 方案流程

object 中对对象各自对应类的状态查询方法及其输出, 如 *obj2.size()* → 5;

常数, 如 0, 1, -1。

3.2 条件表达式期望输出结果收集

当执行完待修复的条件表达式 *exp-patch* 时, 能使所有测试用例能执行通过的值称为条件表达式的期望输出。取值范围为 {true, false}, 用 $O_{l,m,n}$ 来表示期望输出。如果 l 位置为条件语句, 对于测试用例执行是失败的, 天使值即为 *exp-patch* 期望输出, 天使值 (*angelicValue*) 指可以让失败的测试用例执行成功的值, 取值范围为 {true, false}; 如果是成功的测试用例, 条件表达式 *exp-patch* 的实际输出即为期望输出。如果 l 位置为非条件语句, 针对失败的测试用例, $O_{l,m,n}$ 其对应值均为 false, 对于成功的测试用例, $O_{l,m,n}$ 均为 true。

3.3 缺陷修复方程式形成

根据上述实际输入的数据 $I_{l,m,n}$ 和期望的输出结果 $O_{l,m,n}$, 可以形成式 (1) 的待求解的修复方程式 F , 定义为

$$F: \forall I_{l,m,n} \exp(I_{l,m,n}) = O_{l,m,n} \quad (1)$$

所有的 $\langle I_{l,m,n}, O_{l,m,n} \rangle$ 都包含了表达式 *exp-patch* 应该满足的程序语义约束, 可以使用约束求解器对该约束进行求解, 合成候选补丁。

4 基于变异分析技术的程序候选补丁生成

变异分析一般用于检测测试套件对程序缺陷的发现能力^[18], 其思路主要是对原程序 P 中引入小的代码修改, 形成缺陷的变体 P' , 然后运行测试套件 TS , 观察测试套件在原程序 P 和程序变体 P' 中的执行情况, 查看测试套件是否能够检测到引入的缺陷。其中, 引起代码修改的操作被定义为变异算子。

基于文献的研究^[19], 对于条件语句缺陷自动修复, 将变异技术应用其中。如果待修复的表达式 *exp-patch* 为 *if* 条件语句时, 使用预先设计的变异算子, 在该条件表达式中植入小的代码变更, 产生所有可能的一阶程序变体 P'_{bug} , 作为候选补丁。本文依据修复错误的条件三要素: 条件子句、变量和操作符, 设计了如下变异算子。

变异算子 1: 否定条件表达式, 如 *if* ($a > 0 \parallel b < 0$) 变异成 *if* ($a > 0 \parallel b < 0$);

变异算子 2: 替换同类运算符, 包括运算符、关系运算符和逻辑运算符, 如 *if* ($a > 0 \parallel b < 0$) 变异成 *if* ($a > 0 \parallel b > 0$);

变异算子 3: 移除条件表达式中的一个条件子句, 如 *if* ($a < 0 \parallel b < 0$) 变异成 *if* ($a < 0$);

如果一个条件表达式中含有“&&.”或“||”连接的 n 个条件子句, 在不考虑逻辑运算符和子句顺序的情况下, 有 $N = \sum_{m=1}^n C_n^m = \sum_{m=1}^n n! / m!(n-m)!$ 种组合方式。设计变异算子 2 时, 为了有效地控制候选补丁的搜索空间, 只考虑移除表达式中的一个条件子句的情况, 该方式可以产生 n 个程序变体。

在应用变异算子 3 时, 还可以考虑失败的测试用例所对应的天使值, 如果所有失败测试用例的天使值都为 true, 则表示失败测试用例执行完条件表达式的输出应该由 false 转变为 true; 如果都为 false, 则表示失败测试用例执行完条件表达式的输出应该由 true 转变为 false。通过移除一个条件子句来收缩、扩张条件表达式的判定范围, 具体的移除方式为: 收缩判定范围: *if* ($clause \parallel clause_1$) $\geq$ *if* ($clause$)。扩张判定范围: *if* ($clause \&\& clause_1$) $\geq$ *if* ($clause$)。

本文只使用 P_{bug} 的一阶变体作为候选补丁, 比如, 对条件表达式 *if* ($x > 0 \parallel y < 0$) 进行变异, 所有可能的候选

补丁见表 2。

表 2 基于变异算子生成补丁

变异算子	候选补丁
变异算子 1	$if (!x > 0 \parallel y < 0)$
变异算子 2	$if (x < 0 \parallel b < 0)$
	$if (x \leq 0 \parallel y < 0)$
	$if (x \geq 0 \parallel y < 0)$
	$if (x = 0 \parallel y < 0)$
	$if (x \neq 0 \parallel y < 0)$
	$if (x > 0 \parallel y > 0)$
	$if (x > 0 \parallel y \geq 0)$
	$if (x > 0 \parallel y \leq 0)$
	$if (x > 0 \parallel y = 0)$
	$if (x > 0 \parallel y \neq 0)$
	$if (x > 0 \& \& y < 0)$
变异算子 3	$if (x > 0)$
	$if (y < 0)$

由于利用变异技术产生条件表达式的候选补丁时, 可能会生成大量无效的程序变体。为了避免这种情况, 此时, 3.3 节得到的修复方程式可用于初步筛选变异生成的程序变体, 使满足程序语义约束的变体作为候选补丁, 以减少待验证补丁的数量。

5 基于约束求解的程序补丁合成方法

对于上述候选补丁不能修复缺陷的情况, 作为补充本文还提供一种语义驱动的补丁生成方法。利用自动化程序合成技术, 将补丁生成问题转化为约束求解的问题, 将求得解转化为程序补丁。通过运行测试用例, 获取一系列的〈输入, 期望输出〉对, 作为 I/O 的 Oracle; 定义一组用于合成程序的基础组件, 如操作符、常数值、函数方法等; 合成过程以一系列〈输入, 期望输出〉对和基础组件作为输入, 输出一段满足所有〈输入, 期望输出〉对的程序^[9]。

在基于组件合成程序补丁时, 需要用户根据特定的使用场景定义基础组件。本文方案在合成补丁时, 定义的基础组件包含比较运算符 ($>$, $<$, $\neq$, $\leq$, $=$, $\geq$)、算术运算符 ($+$, $-$, $\times$) 和逻辑运算符 ($\wedge$, $\vee$, $\neg$)。其合成过程如下例。

假设待修复的条件表达式 exp_patch 的实际输入数据包含两个变量 x 和 y 以及常数 0, 获取的 I/O Oracle 为 $\langle \{x=1, y=2, 0\}, true \rangle$ 、 $\langle \{x=2, y=1, 0\}, false \rangle$, 可用基础组件 $C = \{+, -, \times, >, <\}$ 。一种有效的合成过程如图 2 所示, 用矩形表示组件, 并用位置号标识 (标签 “loc”), 边表示如何将给定位置生成的值用作在另一位置

的输入 (与每个位置关联的标签 “input”), 式 (1) 生成约束表达式之后, SMT 求解器为与每个位置关联的变量输入找到适当的值。

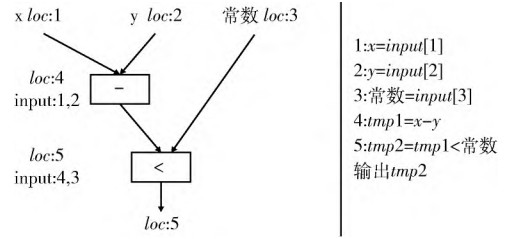


图 2 基于约束求解的程序补丁合成示例

假设合成条件表达式 exp_patch 使用一组基础组件 $\{f_1, f_2, \dots, f_n\}$, f_i 表示第 i 个组件, I_i 表示组件的输入, r_i 表示组件的输出。所有组件的输入集合和输出集合分别为下述 I 和 R

$$I = U_{i=1}^N I_i \quad R = U_{i=1}^N \{r_i\}$$

待求解条件表达式的一组实际输入为 I_0 (即 $I_{l,m,n}$), 对期望输出为 r (即 $O_{l,m,n}$), 所有的输入输出集合为

$$IO = \{I \cup R \cup I_0 \{r\}\}$$

定义位置变量

$$LOC = \{loc_x \mid x \in IO\}$$

loc_x 表示 IO 中的每个元素 x 的索引位置; 取值变量

$$V = \{v_x \mid x \in IO\}$$

v_x 表示 IO 中的每个元素 x 的取值。对于每个测试用例的每次执行, 位置变量代表了补丁的组成结构, 应该保持一致。取值变量则被 SMT 求解器使用, 用于保证合成的程序符合 I/O oracle。

LOC 应该保证语法约束和语义约束^[7]。语法约束规定了在程序语法结构上待求解的条件表达式 exp_patch 应该满足的要求, 其定义如式 (2) 所示

$$\phi_{wfp}(LOC, I, I_0, R, r) = \phi_{fixed}(I_0, r) \wedge \phi_{output}(R) \wedge \phi_{input}(I) \wedge \phi_{cons}(LOC, R) \wedge \phi_{acyc}(LOC, I, R) \quad (2)$$

式中: ϕ_{fixed} 该约束表示对于待求解条件表达式的实际输入 I_0 中的第 i 个元素, 其位置变量 loc 的值为 i ; 对于待求解条件表达式的期望输出 r 中的元素, 其位置变量 loc 的值 M 。 $\phi_{output}(R)$ 该约束限制一个基础组件 f_i 所定义的中间变量 r ; 其位置变量 loc , 应该在 $|I_0| + 1$ 到 M 之间。 $\phi_{input}(I)$ 表示不同的基础组件能够处理不同类型的数据, 该约束对每个基础组件所使用的数据 x 的数据类型进行限制。定义 $type(x)$ 方法可以返回与 x 具有相同数据类型的数据, 基础组件的输入数据 x 只能取自于这些数据, 即 $LOC_x = LOC_y$ 。 $\phi_{cons}(LOC, R)$ 该约束表示每个基础组件所定义的变量都应该有不同的位置变量, 以便在其中使用该变量时, 可以索引到。 $\phi_{acyc}(LOC, I, R)$ 该约束则限制了在同一个位置的元素应该具有相同的数据值。

除了语法约束之外, LOC 还应满足一定的语义约束。其定义如式 (3) 所示

$$\phi_{func}(LOC, I_0, r) = \phi_{lib}(I, R) \wedge \phi_{comm}(LOC, I_0, r, I, R) \quad (3)$$

式中: $\phi_{lib}(I, R)$ 该约束表示任意一个基础组件的输入 v_{I_i} 经过基础组件所定义的运算 f_i 后, 输出值为 v_{r_i} , $\phi_{comm}(LOC, I_0, r, I, R)$ 该约束限制了在同一个位置的元素应该具有相同的数据值。

结合语法约束和语义约束, 对于 I/O oracle 中所有的 $\langle$ 输入, 期望输出 $\rangle$ 对, LOC 需要满足的完整约束如式 (4) 所示

$$\theta = \wedge(I_{l,m,n}, O_{l,m,n}) (\phi_{wfp}(LOC, I, I_0, R, r) \wedge \phi_{func}(LOC, I_0, r) \wedge (I_{l,m,n} = I_0) \wedge (O_{l,m,n} = r)) \quad (4)$$

利用 SMT 求解器对约束 θ 求解, 若 LOC 为该约束的解, 则根据 LOC 和定义的基本组件, 可以构造出满足程序约束的条件表达式 $exp-patch$ 。

如果可修复程序位置为 if 条件语句, 则直接使用表达式 $exp-patch$ 对原始表达式进行替换, 形成候选的程序补丁; 如果为非 if 条件语句, 则将表达式 $exp-patch$ 作为前置条件添加, 形成候选补丁。

6 实验验证

实验基于上述问题设计实现了一个条件语句缺陷自动修复工具原型 MSFix, 进行方案的可行性和有效性验证。

实验对象选择 Defects4 J^[20] 数据集集中的 14 个 Java 项目的 62 个条件语句缺陷, 其中条件语句错误缺陷 41 个, 条件语句缺失缺陷 21 个, 见表 3。

实验采用召回率和准确率来评价缺陷修复效果。在修复一个缺陷的过程中, 如果能够找到一个有效补丁能通过所有的测试, 则认为该缺陷被成功修复, 召回率计算方式如式 (5) 所示

$$\text{召回率} = \frac{\text{被修复的缺陷数量}}{\text{缺陷总数}} \times 100\% \quad (5)$$

在修复缺陷的过程中, 如果一个有效补丁正确修复了缺陷, 则认为该补丁是正确的补丁, 补丁准确率计算方式如式 (6) 所示

$$\text{准确率} = \frac{\text{正确补丁数量}}{\text{有效补丁数量}} \times 100\% \quad (6)$$

6.1 可行性实验——Java 程序中条件语句缺陷的修复

通过运行 MSFix, 对上述的缺陷进行修复, 返回 MSFix 的修复结果。如果缺陷修复成功, 将 MSFix 修复的补丁与 Defects4 J 提供的补丁进行比较, 验证补丁的有效性。由表 4 的实验结果可知, 在上面提供的项目中 69 个条件语句缺陷来说, MSFix 能够修复 39 个缺陷, 并且整体的召回率为 62.90%。

在 62 个条件语句缺陷中, 也有 23 个缺陷没有修复成功。分析其修复失败的原因主要是:

表 3 实验对象信息

项目名称	项目 ID	条件语句缺陷数	条件语句错误缺陷编号	条件语句缺失缺陷编号
jfreechart	Chart	6	1, 5, 9	4, 15, 26
commons-lang	Lang	8	15, 16, 36, 58	3, 46, 55, 65
commons-math	Math	13	15, 26, 32, 33, 37, 43, 82, 85, 94, 101	28, 84, 95
mockito	Mockito	2	34	15
joda-time	Time	2	无	3, 27
commons-cli	Cli	1	11	无
commons-codec	Codec	1	2	无
commons-compress	Compress	5	19, 20, 21, 38	11
commons-csv	Csv	3	1, 14	5
jackson-core	Jackson-Core	2	14	21
gson	Gson	4	13, 15	6, 12
jackson-databind	Jackson-Databind	9	8, 27, 28, 45, 54, 71	49, 65, 80
jsoup	Jsoup	3	83, 86, 90	无
commons-jxpath	JXPath	3	6, 17, 19	无
(缺陷个数)		62	41	21

表 4 MSFix 的整体实验结果

项目名称	缺陷数目	被修复的缺陷数量 (有效补丁数量)	召回率/%
Chart	6	6	100.00
Lang	8	4	50.00
Math	13	8	61.54
Mockito	2	0	0.00
Time	2	1	50.00
Cli	1	1	100.00
Codec	1	1	100.00
Compress	5	4	80.00
Csv	3	1	33.33
JacksonCore	2	2	100.00
Gson	4	3	75.00
JacksonDatabind	9	5	55.56
Jsoup	3	2	66.67
JXPath	3	1	33.33
合计	62	39	62.90

(1) 由于本文方案输入的可疑语句是通过可疑分数进行定位的, 可疑语句定位时会计算可疑分数阈值, 并舍弃

可疑分数低于阈值的程序方法, 只对剩余方法中的程序语句进行分析。由于部分缺陷 (如 Math-26) 的测试用例不够充足, 缺少失败测试用例对错误语句的覆盖, 导致错误语句所在的程序方法的可疑分数较低。在进行可疑语句定位时, 含有错误语句的程序方法被舍弃, 最终可疑语句列表中并没有真正错误的语句。

(2) 由于本文方案输入的可疑语句是通过缺陷定位得到的。在定位可修复程序位置过程中, 期望每个失败的测试用例都存在一个使其能够执行通过的天使值。由于部分缺陷 (如 Lang-16) 的一个失败测试用例中存在多个 assert 断言, 这些 assert 断言覆盖了 if 结构的 then 分支和 else 分支。在使用 true/false 替换 if 条件表达式后, 重新运行这个失败的测试用例, 没有办法同时满足多个 assert 断言。因此, 无法找到这个失败测试用例的天使值, 导致可修复程序位置定位失败。通过以上分析可知, 测试用例的质量不高是导致缺陷修复失败的重要原因。针对情况 (1), 可以通过额外增加失败的测试用例来提高错误语句的覆盖率, 使其能够出现在可疑语句列表中, 或者排名更靠前。针对情况 (2), 可以通过拆分失败的测试用例, 使每个测试用例中只有一个 assert 断言, 在可修复程序位置定位中, 更可能找到使失败测试用例都通过天使值。

6.2 有效性实验——缺陷修复效果与其它方法的对比

目前针对修复 Java 程序缺陷的方案中, Nopol^[11,12]、jGenProg^[3]和 jKali^[5]这 3 种工具比较有代表性, 在比较多的相关研究中引用。因此, 在方案有效性实验中, 选择这 3 种工具作为比较对象。对上述选择的 62 个条件语句缺陷, 分别应用上述的 3 个工具和 MSFix 进行自动修复, 记录并分析每个工具的修复效果, 计算每个工具的缺陷召回率和准确率, 并进行分析。表 5 展示了 4 种方法的整体修复结果。可以看出 MSFix 和上面 3 种修复工具相比, 其修复的正确补丁数量是最多的, 一共修复了 21 个, 而其它 3 个工具却修复没有超过有效补丁数量的一半。并且 MSFix 的召回率和准确率都分别达到了 62.90% 和 53.85%。其它 3 种工具的召回率和准确率都比较偏低。

表 5 4 种工具对比的整体实验结果

修复工具	修复的缺陷数量	正确补丁数量	召回率/%	准确率/%
MSFix	39	21	62.90	53.85
Nopol	25	9	40.32	36.00
jGenProg	12	3	19.35	25.00
jKail	14	2	22.58	14.29

使用 Venn 图来展示这 4 个工具所修复缺陷之间的交叉重叠情况。由表 5 和图 3 可知, jGenProg 和 jKali 的召回率相对较低, 并且被 jGenProg 和 jKali 这两个工具修复的缺陷

都能够被 Nopol 和 MSFix 修复。MSFix 和 Nopol 相比, MSFix 能够修复 17 个 Nopol 无法修复的缺陷, 召回率提高了 22.58%。

未修复的缺陷: 20

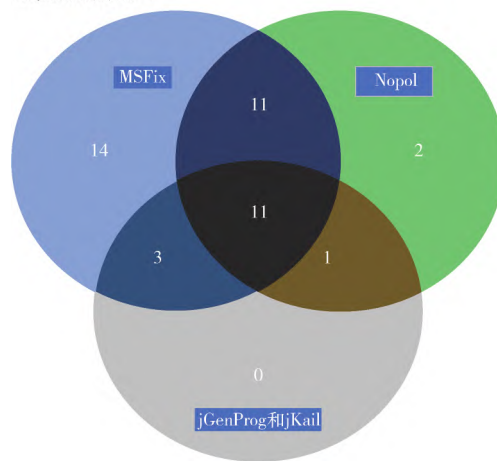


图 3 4 个工具修复缺陷韦恩

由于 jGenProg 和 jKali 在基于各自定义的操作修改缺陷程序形成候选补丁时, 对所有类型的缺陷都执行相同的修复操作。虽然能够修复条件语句缺陷, 但相比于 MSFix 和 Nopol, jGenProg 和 jKali, 缺少更有针对性的分析。因此, 在修复条件语句缺陷时, jGenProg 和 jKali 生成的有效补丁相对较少, 缺陷修复较低。另外, Nopol 在生成候选补丁时, 只使用了基于约束求解的补丁生成方法。本文方案则在此基础上, 增加、并且优先使用变异技术生成程序补丁, 综合使用两种补丁生成方法, 使得对条件语句缺陷具有更高的修复率。

在补丁准确率方面, 4 个工具都基于测试套件进行修复, 但本文方法在生成程序补丁时, 优先考虑使用变异技术对原始表达式进行变异, 基于真实的缺陷修复记录, 为条件语句定义合适的变异算子, 该方法更可能产生符合开发人员预期的程序补丁。因此, MSFix 能够优先输出正确的程序补丁, 补丁准确率相对其它 3 种方法都有所提高。

7 结束语

缺陷自动修复方法是帮助开发人员减少调试成本的有效技术, 为了修复 Java 程序中常见的条件语句缺失缺陷和条件语句逻辑错误缺陷, 本文提出了基于变异分析和约束求解的程序补丁合成技术, 实验结果表明本文方案能够修复真实的条件语句缺陷, 并且相比于现有修复方法, 本文方案具有更好的修复效果。在相当程度上解决了 Java 程序中条件语句缺陷的自动修复问题。但本文的研究基于 Java 程序, 对其它语言的适应性还有待进一步的实验验证。在变异分析中, 只采取了 3 种变异算子, 在未来工作中, 可以考虑增加更多的变异算子, 以提高缺陷修复的能力; 此

外实验结果表明本方法修复的补丁和现有的工具相比, 修复的补丁既有重叠的又有独有的, 为了提高缺陷自动修复的可靠性和完备性, 在未来工作中, 可以考虑综合使用补丁修复的工具, 对本文方法做出进一步的改进, 以提高补丁修复率。

参考文献:

- [1] ISO/IEC/IEEE international standard-systems and software engineering—Vocabulary: ISO/IEC/IEEE 24765: 2017 (E) [S/OL]. [2017-08-28]. <https://ieeexplore.ieee.org/servlet/opac?punumber=8016710>.
- [2] JIANG Jiajun, CHEN Junjie, XIONG Yingfei. Survey of automatic program repair techniques [J] Journal of Software, 2021, 32 (9): 2665-2690 (in Chinese). [姜佳君, 陈俊洁, 熊英飞. 软件缺陷自动修复技术综述 [J]. 软件学报, 2021, 32 (9): 2665-2690.]
- [3] Matsumoto J, Higo Y, Matsuo H, et al. Kusumoto: Gen-Prog meets cluster computing [C] //10th International Workshop on Empirical Software Engineering in Practice, 2019: 37-42.
- [4] Ye H, Martinez M, Durieux T, et al. A comprehensive study of automatic program repair on the quixbugs benchmark [C] // IEEE 1st International Workshop on Intelligent Bug Fixing, 2019: 1-10.
- [5] Martinez M, Monperrus M. ASTOR: A program repair library for Java [C] //Proceedings of the 25th International Symposium on Software Testing and Analysis, 2016: 441-444.
- [6] Gazzola L, Micucci D, Mariani L. Automatic software repair: A survey [C] //IEEE Transactions on Software Engineering, 2019, 45 (1): 34-67.
- [7] Nguyen H, Qi D, Roychoudhury A, et al. SemFix: Program repair via semantic analysis [C] //Proceedings of the International Conference on Software Engineering, 2013: 772-781.
- [8] Le X B D, Chu D H, Lo D, et al. JFIX: Semantics-based repair of Java programs via symbolic PathFinder [C] //Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2017: 376-379.
- [9] Jha S, Gulwani S, Seshia S A, et al. Oracle guided component-based program synthesis [C] //Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering, 2010: 215-224.
- [10] Mechtaev S, Yi J, Roychoudhury A. Angelix: Scalable multiline program patch synthesis via symbolic analysis [C] // Proceedings of the 38th International Conference on Software Engineering, 2016: 691-701.
- [11] Wang Shangwen, Wen Ming, Mao Xiaoguang, et al. Attention please: Consider mockito when evaluating newly proposed automated program repair techniques [C] //Proceedings of the Evaluation and Assessment on Software Engineering. Association for Computing Machinery, 2019: 260-266.
- [12] Xuan J, Martinez M, DeMarco F, et al. Nopol: Automatic repair of conditional statement bugs in Java programs [J]. IEEE Transactions on Software Engineering, 2017, 43 (1): 34-55.
- [13] van Tonder R, Le Goues C. Static automated program repair for heap properties [C] //Proc of the 40th Int'l Conf on Software Engineering. ACM, 2018: 151-162.
- [14] Xiong Y, Wang J, Yan R, et al. Precise condition synthesis for program repair [C] //IEEE/ACM 39th International Conference on Software Engineering. IEEE, 2017: 416-426.
- [15] Long F, Amidon P, Rinard M. Automatic inference of code transforms and search spaces for automatic patch generation systems [C] //Joint Meeting on Foundations of Software Engineering. ACM, 2017: 727-739.
- [16] WANG Zan, GAO Jian, CHEN Xiang, et al. Automatic program repair techniques: A survey [J]. Chinese Journal of Computers, 2018, 41 (3): 588-610 (in chinese). [王赞, 郜健, 陈翔, 等. 自动程序修复方法研究述评 [J]. 计算机学报, 2018, 41 (3): 588-610.]
- [17] Liu K, Koyuncu A, Kim D, et al. AVATAR: Fixing semantic bugs with fix patterns of static analysis violations [C] //IEEE 26th International Conference on Software Analysis, Evolution and Reengineering, 2019: 1-12.
- [18] Du Y, Pan Y, Ao H, et al. Automatic test case generation and optimization based on mutation testing [C] //IEEE 19th International Conference on Software Quality, Reliability and Security Companion, 2019: 522-523.
- [19] Thomas Helmuth, Nicholas Freitag McPhee, Lee Spector. Program synthesis using uniform mutation by addition and deletion [C] //Proceedings of the Genetic and Evolutionary Computation Conference. Association for Computing Machinery, 2018: 1127-1134.
- [20] Sobreira V, Durieux T, Madeiral F, et al. Dissection of a bug dataset: Anatomy of 395 patches from Defects4J [C] // IEEE 25th International Conference on Software Analysis, Evolution and Reengineering, 2018: 130-140.